Stencils at the Roofline from Python: the Heat Equation and Jacobi Iteration on a Laptop GPU, and How Temporal Blocking Moves the Roof

Walid Hemati
Independent researcher

Abstract

The explicit heat equation with the 7-point (3D) or 5-point (2D) stencil, and the Jacobi iteration that shares its update, are the textbook memory-bound kernels: about one floating point operation per byte of DRAM traffic. We use them to test a simple claim: Python code on a GPU reaches the same *hardware efficiency* as hand-written CUDA C. One package, `pyheat3d`, implements the update in NumPy, PyTorch eager and `torch.compile`, Triton, NVIDIA Warp, Numba-CUDA and hand-written CUDA C compiled at run time through CuPy, all behind one interface and validated against analytic solutions (second order in space, first order in time, to the expected rates) and against a NumPy oracle. On an RTX 3070 Laptop GPU the best single-step kernel reaches 41.0 GLUPS on a 512^3 grid, which is 91% of the measured 361 GB/s copy bandwidth under a minimal 8-byte-per-update traffic model. We then add temporal blocking, which fuses k time steps into one DRAM pass. This does not beat the hardware. It lowers the traffic per update k-fold and so raises the per-step roofline by the same factor. With it the CUDA C variant reaches 94.1 GLUPS in 3D (2.09× the single-step roofline) and 157.8 GLUPS in 2D. We report both lattice updates per second and the estimated DRAM traffic behind them.

1 Introduction

Stencil codes sweep a grid and update each site from a few neighbours. They do very little arithmetic per byte moved, so on modern hardware their speed is set by memory bandwidth, not by floating point throughput. The roofline model [1] makes this exact: a kernel with arithmetic intensity I (FLOP/byte) cannot exceed $\min(P, I \cdot B)$, where P is peak compute and B peak bandwidth. For the explicit heat equation in single precision, $I \approx 1$ FLOP/byte and the bandwidth term dominates by more than an order of magnitude.

That makes the heat equation a fair test for high-level languages. There is no algorithmic headroom to hide behind and no tensor core to borrow. Whatever reaches the copy bandwidth has done the job, whether it is Python or C. This paper asks how close each mainstream Python GPU route gets, what it costs in code, and what happens when the algorithm itself changes the roof.

Contributions.

- An optimisation ladder for the same stencil in seven backends (NumPy, PyTorch eager, `torch.compile`, Triton, Warp, Numba-CUDA, CUDA C via CuPy/NVRTC), each as a variant in a common registry with one interface, one test suite and one benchmark harness.
- Validation that does not depend on any backend: the decaying Fourier mode against its continuous, semi-discrete and fully discrete exact solutions, Laplace and Poisson steady states, and 76/76 cross-backend comparisons against a NumPy oracle.

- A roofline analysis with a stated minimal traffic model. Every throughput is reported as lattice-site updates per second and as a percentage of the measured copy bandwidth (361 GB/s) and of the specification (448 GB/s).
- Temporal-blocking kernels written from Python (3D: shared memory plus warp shuffles with register blocking; 2D: warp-synchronous, no shared memory). They exceed the single-step roofline, and we explain exactly why this is not the same as exceeding the hardware.
- A same-GPU head-to-head with two established stencil codes, the pystencils code generator and AN5D's temporal-blocking CUDA, on the identical update.

2 Background and related work

Roofline. Williams, Waterman and Patterson [1] bound attainable performance by peak compute and by bandwidth times operational intensity, where intensity counts bytes of DRAM traffic. The bound depends on the *traffic*, not on the FLOPs alone. Any transformation that cuts traffic per unit of work raises the ceiling for that work.

Stencil optimisation. Datta et al. [2] auto-tuned 7-point stencils across multicore architectures and showed they are bandwidth bound once cache blocking removes redundant loads. On GPUs, Micikevicius [3] introduced the now standard 2.5D scheme: a thread block holds an xy tile in shared memory and marches along z, keeping the z neighbours in registers. Nguyen et al. [4] combined 2.5D spatial blocking with temporal blocking ("3.5D") on CPUs and GPUs. Holewinski, Pouchet and Sadayappan [5] generated GPU stencil code with overlapped tiling, recomputing halos redundantly instead of synchronising between blocks. pystencils, the code generator behind waLBerla [6, 7], turns symbolic stencil updates written in Python into CPU and GPU kernels.

Temporal blocking. Wonnacott's time skewing [8] reorders the iteration space so that several time steps run on data still in cache. That removes the bandwidth limit in exchange for more work per byte. Wellein et al. [9] pipelined successive time steps across cores sharing a cache (wavefront parallelisation). Malas et al. [10] used diamond tiling with multicore wavefronts. Our temporally blocked kernels are a GPU form of the wavefront idea: a thread block streams through z and k pipeline stages advance the same columns by k time levels, using overlapped (redundant) halos as in [5]. AN5D [11] generates GPU code with temporal blocking degrees up to 10 from C loop nests, guided by a performance model.

Python on GPUs. NumPy [12] defines the array model the others imitate. PyTorch 2's `torch.compile` [13] captures Python into a graph and generates fused Triton kernels. Triton [14] lets one write block-level kernels in Python and leaves intra-block scheduling to the compiler. Warp [15] and Numba [16] compile SIMT kernels written in a Python subset. CuPy [17] provides NumPy-compatible arrays and runtime compilation of CUDA C through NVRTC. We use the last route as our "hand-written CUDA C" reference: the kernels are compiled at run time by NVRTC, so the package needs no CUDA toolkit or `nvcc` at install time.

3 Method

3.1 Physics and numerics

We solve $u_t = \alpha \nabla^2 u$ on the unit square or cube with forward Euler in time and second-order central differences in space (FTCS). With uniform spacing h and the mesh Fourier number $r = \alpha \Delta t / h^2$ the update is

$$u'_c = (1 - 2dr)\, u_c + r\, S_c, \qquad S_c = \big[(u_{x-1} + u_{x+1}) + (u_{y-1} + u_{y+1})\big] + (u_{z-1} + u_{z+1}), \tag{1}$$

where $d \in \{2, 3\}$. The von Neumann limit is $r \le 1/(2d)$, which the code enforces. Every backend uses the same bracketing of S_c, so they differ at most by fused multiply-add contraction. Dirichlet problems keep a fixed boundary layer that no kernel writes; periodic problems wrap indices. The Jacobi iteration for $\nabla^2 u = f$ is the same update with $c_0 = 0$, $c_1 = 1/(2d)$ and an additive term $-h^2 f/(2d)$. All kernels are written for the generic $u' = c_0 u + c_1 S$ and run both modes.

3.2 Reference solutions

For the mode $u_0 = \prod_i \sin(\kappa x_i)$ ($\kappa = \pi$ Dirichlet, 2π periodic) three exact references exist: (i) the PDE solution $e^{-\alpha d \kappa^2 t} u_0$; (ii) the semi-discrete (exact-in-time) solution $e^{-\alpha \lambda_h t} u_0$ with $\lambda_h = 4d \sin^2(\kappa h/2)/h^2$; and (iii) the fully discrete solution $g^n u_0$ with $g = 1 - 4dr \sin^2(\kappa h/2)$, which FTCS must reproduce to round-off. Error against (i) at fixed r measures the combined $\mathcal{O}(h^2) + \mathcal{O}(\Delta t) = \mathcal{O}(h^2)$ error. Error against (ii) on a fixed grid isolates the first-order temporal error. Error against (iii) is a pure implementation check. We avoid $r = 1/(6d)$, where the leading space and time error terms cancel.

For Laplace we use $\sin(\pi x)\sinh(\pi y)/\sinh \pi$ (2D) and $\sin(\pi x)\sin(\pi y)\sinh(\sqrt{2}\pi z)/\sinh(\sqrt{2}\pi)$ (3D), and quadratic harmonic functions on which the discrete Laplacian is exact. For Poisson we use a manufactured solution $u = \prod \sin(\pi x_i)$.

4 Implementation

4.1 One interface, a registry of variants

Each variant is a `Stepper` that owns two device buffers and exposes `step(n)`, `get()` and `synchronize()`. A registry entry records backend, dimensionality, storage type and k, the number of time steps fused per DRAM pass. Tests, validation and benchmarks loop over the registry. Variants whose package or GPU is missing are skipped. Table 1 lists the ladder.

4.2 Traffic model and roofline

We use the *minimal* traffic model: a sweep that advances the field by k steps must read each value once and write it once, so the DRAM traffic per lattice-site update is

$$B_{\text{upd}} = \frac{2\,b}{k}, \qquad b = \text{bytes per stored value.} \tag{2}$$

That is 8 B for fp32 single-step kernels and 4 B for fp16/bf16 storage. Reported effective bandwidth is LUPS $\times\, B_{\text{upd}}$. Halo re-reads and cache misses are not counted, so this is a lower bound on true traffic; an inefficient kernel cannot inflate it. With the measured 361 GB/s copy bandwidth the single-step fp32 roofline is $361/8 = 45.1$ GLUPS and the 2-byte roofline is 90.2 GLUPS. The update needs $2d + 2$ FLOPs (eq. (1)), so $I = 1$ FLOP/byte in 3D: at least 25× below the ridge point of this GPU.

Table 1: The optimisation ladder. k: time steps per DRAM pass.

backend	variant	k	idea
numpy	slicing	1	vectorised slices, CPU; correctness oracle
torch-eager	slicing	1	same slices on the GPU; one kernel per operator
torch-compile	inductor	1	the same Python traced and fused into one Triton kernel
triton	tile(-fp16/-bf16)	1	2D/3D tiles, neighbours as shifted masked loads
triton	zmarch(-fp16/-bf16)	1	3D: xy tile marches in z with a register queue
warp, numba	naive, zmarch	1	one thread per site; per-thread z column with register queue
cupy-cuda-c	naive(-fp16/-bf16)	1	one thread per site (several per thread for 2-byte types)
cupy-cuda-c	zmarch, ymarch	1	2.5D: shared-memory plane + register queue [3]
cupy-cuda-c	tb2, tb3, tb4, tb8	k	temporal blocking (wavefront, overlapped halos)
cupy-cuda-c	tb4-fp16	4	temporal blocking with fp16 storage

What temporal blocking does to the roof. For a k-step kernel eq. (2) gives $8/k$ bytes per fp32 update, so its roofline is $k \cdot 45.1$ GLUPS. A temporally blocked kernel that delivers more than 45.1 GLUPS has not moved data faster than the hardware allows. It moves k times less data per update. For these kernels we therefore report three numbers: the update rate; the DRAM traffic implied by eq. (2) (always below the copy bandwidth); and the "single-step-equivalent bandwidth" LUPS $\times$ 8 B. The last one is *not* a DRAM rate, only the bandwidth a $k = 1$ kernel would need to match the rate. For the CUDA C kernels we also give a more realistic *halo model* that adds the redundant halo reads of overlapped tiling (section 4.3), assuming no L2 reuse between blocks.

4.3 Kernels

Tiles, compiled expressions and naive SIMT. The Triton tile kernels load the centre tile and its four or six shifted neighbours as masked loads. Reuse between neighbouring loads is left to L1/L2. `torch.compile` receives the NumPy-style expression of eq. (1) with slicing (Dirichlet) or `torch.roll` (periodic) and lowers it to one fused kernel. Warp, Numba and the naive CUDA C kernels use one thread per site. For 2-byte storage the CUDA C kernel handles several sites per thread, to keep enough bytes in flight.

2.5D z-marching. Following [3], the CUDA C `zmarch` kernel stages the current z plane plus a one-site halo in shared memory and keeps the $z \pm 1$ values in a register queue. It prefetches the next plane one iteration ahead. The Triton, Warp and Numba `zmarch` variants keep the register queue and leave x/y reuse to the caches.

Temporal blocking. The 3D kernel assigns a $32 \times T_y$ thread block to an extended xy tile of $32 \times (T_y R)$ columns. Lane x owns a column and each thread owns R consecutive y rows (register blocking). The block streams through the planes of a z chunk. At iteration i_z it loads level t of plane i_z, and stage $s = 1..k$ computes level $t + s$ of plane $i_z - s$ from a register queue of level $t + s - 1$. That is a wavefront in (z, t), as in [9], mapped onto one thread block. The x neighbours come from warp shuffles, the y neighbours from the thread's own registers, and only the first and last row of each thread pass through shared memory, with one barrier per stage. The halo is recomputed redundantly (overlapped tiling [5]). The valid region shrinks by one column per stage,

and only the central $(32 - 2k) \times (T_y R - 2k)$ columns are stored, once per k steps. The 2D kernel is warp-synchronous: each warp owns an x strip of $32R$ columns, with lane l holding columns $l, l + 32, \ldots$ so every load is coalesced. It marches along y with no shared memory and no barriers. The chunk length along the marching direction is chosen at run time so small grids still fill the GPU. Intermediate levels never leave registers, so the fp32 kernels give the same arithmetic as k single steps and the fp16 kernel rounds to storage only once per k steps.

All CUDA C kernels are one source file compiled by NVRTC with macros for storage type, boundary condition, k and tile shape. The Python side picks the launch configuration. The backends together are about 1,400 lines of Python and CUDA C, comments included.

5 Validation

Figure 1 summarises the studies. They run with the fp64-capable variant `torch-eager/slicing`, except panel (d). Against the PDE solution at fixed $r = 0.1$ the observed spatial order is 1.97 (2D) and 1.99 (3D) with Dirichlet walls, and 2.00/2.08 on periodic grids. The finest 3D grid (129^3) has relative error 2.4×10^{-5}. Against the semi-discrete solution on a fixed grid the temporal order is 1.00 (2D, $n = 65$) and 1.00 (3D, $n = 33$). The converged Jacobi solutions converge to the smooth harmonic functions with order 1.99 (2D) and 1.98 (3D). The manufactured Poisson problems converge with 2.00 and 2.01. On the quadratic harmonic function, where the stencil is exact, Jacobi recovers the solution to 5.3×10^{-15} (3D), which is round-off and iteration tolerance only.

Every GPU variant reproduces the fully discrete FTCS solution of the Fourier mode to at most 1.2×10^{-5} in 3D (fp32, 65^3, 410 steps), while its error against the PDE is 6.0×10^{-5}. Round-off is therefore far below discretisation error. On random fields with both boundary conditions, 76 of 76 comparisons of 23 variants against the NumPy oracle pass. The worst fp32 relative deviation is 5.1×10^{-7}, which comes from FMA contraction only. fp16 and bf16 variants are compared with a NumPy reference that applies the same storage rounding after every DRAM pass (worst 8.9×10^{-4} and 7.1×10^{-3}). That makes the test exact up to a few storage ulps.

Storage precision matters for physics: on the decaying mode the fp16 and bf16 single-step variants reach 7.3×10^{-2} and 3.5×10^{-1} against the PDE in 3D, against 6.0×10^{-5} in fp32, because each step's small increment $r\nabla^2 u$ is rounded to an 8- or 11-bit mantissa. Temporal blocking with fp16 storage rounds only every fourth step and lands at 7.8×10^{-4}. Low-precision storage halves the roofline cost, but for long diffusive runs bf16 is not accurate enough.

6 Performance

All numbers come from `bench/run.py` (`results/rtx3070-laptop.json`, 186 runs, NVIDIA GeForce RTX 3070 Laptop GPU, driver 595.91.07, PyTorch 2.7.0+cu126, Triton 3.3.0, Warp 1.18.0, CuPy 13.5.1, Numba 0.68.0, Python 3.13.2). Each run warms up (including JIT compilation), synchronises the device around the timed region and reports the median of five repeats, each long enough to be timed reliably, together with min and max. Dirichlet heat mode at $r = 0.9/(2d)$, fp32 unless stated.

Single step: Python at the roof. In 3D (512^3) the best single-step variant, `numba/zmarch`, runs at 41.0 GLUPS (min–max 39.1–41.1). That implies 328 GB/s of DRAM traffic: 91% of the measured copy bandwidth and 73% of the specification. In 2D (16384^2) the best is `torch-compile/inductor` with 45.5 GLUPS (101% of copy). Figure 2 and table 2 show which Python routes reach the level

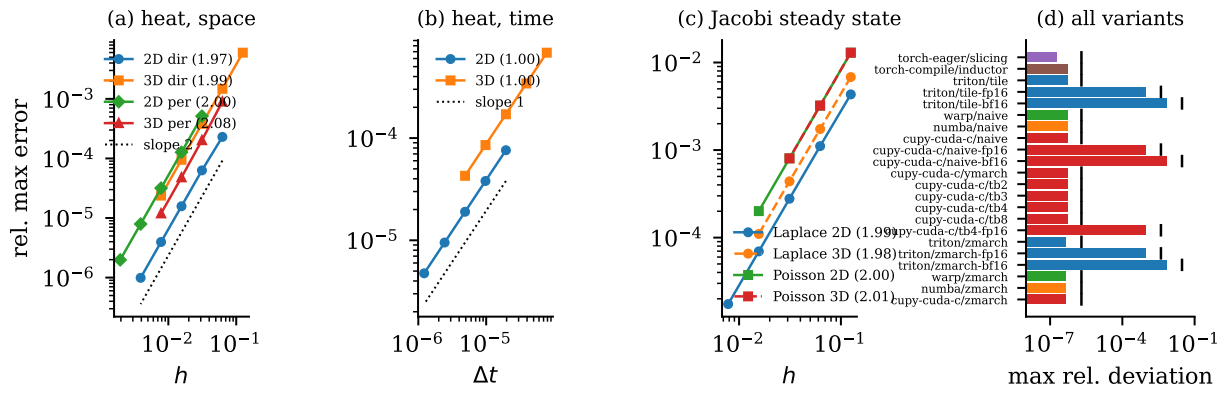


Figure 1: Validation. (a) Error against the exact PDE solution of a decaying sine mode at fixed $r = 0.1$ (2nd order). (b) Error against the semi-discrete solution on a fixed grid (1st order in Δt). (c) Jacobi steady states for Laplace and Poisson. (d) Worst relative deviation of every GPU variant from the (storage-emulating) NumPy reference; ticks mark the tolerance. Observed orders in the legends.

of the hand-written CUDA C (97% of copy in 2D, 89% in 3D). In 2D, `torch.compile` (101%) and Triton (100%) do; the plain Warp and Numba kernels stop at 71% and 76%. In 3D the z-marching kernels in Numba (91%), Warp (89%) and Triton (85%) do, while `torch.compile` reaches 58%. Where they reach it, the Python routes differ more in small-size launch overhead (fig. 5) than in large-size bandwidth. PyTorch eager is far below, for a structural reason: each of its roughly ten operators is a separate pass over memory. The same Python expression handed to `torch.compile` runs as one kernel. Against NumPy on the 16-core CPU the best GPU kernel is 72× faster in 3D at 64^3.

Low-precision storage. fp16/bf16 storage halves B_{upd}. The best 2-byte single-step variant in 3D, `triton/zmarch-bf16`, reaches 78.0 GLUPS, that is 312 GB/s or 86% of copy bandwidth. 2-byte kernels need more independent loads in flight per thread to saturate DRAM, so they sit further below their own roof than fp32 kernels do below theirs.

Temporal blocking: a higher roof, not faster DRAM. The CUDA C kernel `cupy-cuda-c/tb3` ($k = 3$) delivers 94.1 GLUPS in 3D, 2.09× the single-step fp32 roofline. Under the minimal model it moves 251 GB/s (70% of copy). With the halo over-fetch of overlapped tiling the estimate is 333 GB/s (3.54 B per update). A single-step kernel would need 753 GB/s of DRAM bandwidth to match the rate (the specification is 448 GB/s). In 2D, `cupy-cuda-c/tb8` ($k = 8$) reaches 157.8 GLUPS (3.50× the single-step roof) while using 158 GB/s. The gain saturates with k: redundant halo work grows as $(T - 2k)^{-1}$ per tiled dimension, the pipeline holds $3k$ values per site in registers, and in 3D every stage costs a barrier. At large k these kernels become instruction-bound rather than bandwidth-bound. Figure 4 shows the same data on the classic roofline. Temporal blocking moves points to the right (higher intensity); it does not move them above the memory roof.

6.1 Head-to-head with established stencil codes

To check our kernels against code written by others, `bench/counterparts.py` runs two established stencil codes on the same GPU, in the same session as our own variants, with the same update (eq. (1), fp32, Dirichlet, $r = 0.9/(2d)$). *pystencils* [6] (version 2.1, CUDA target through CuPy) is

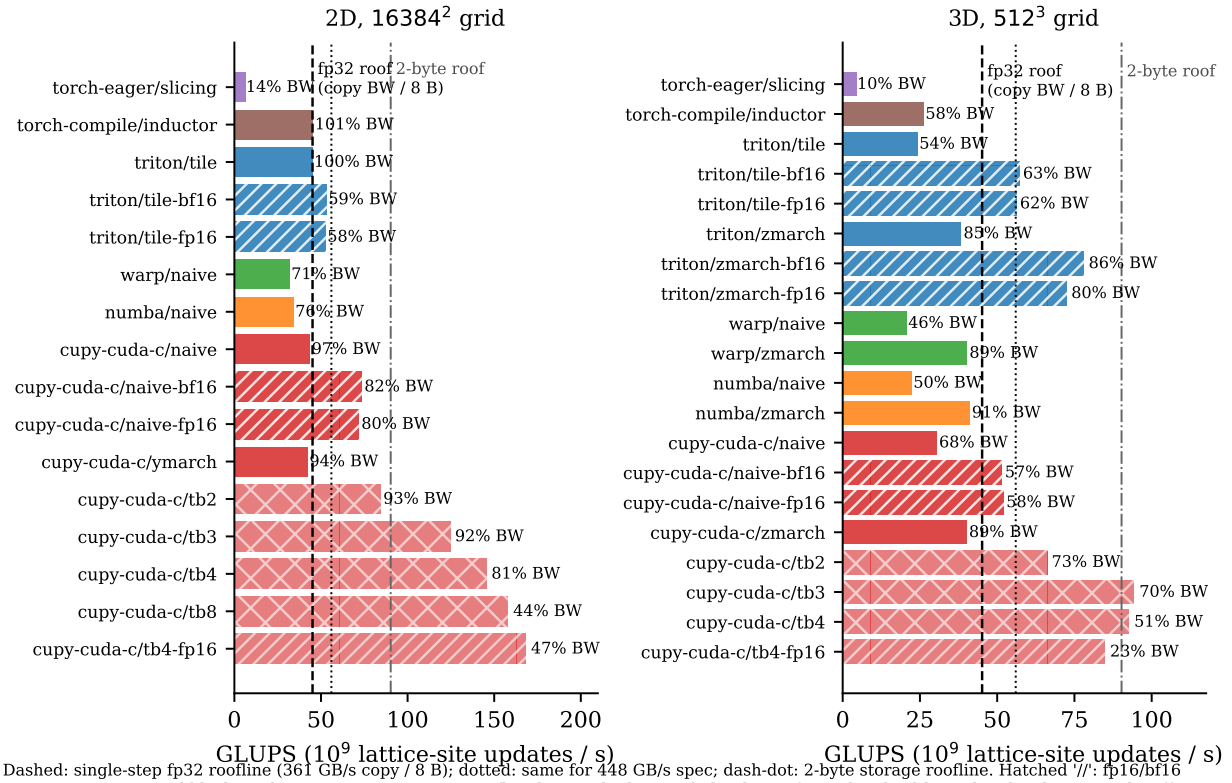


Figure 2: Throughput of every variant at the largest benchmarked size, against the fp32 single-step roofline (dashed, copy bandwidth / 8 B), the same at the specification bandwidth (dotted) and the 2-byte roofline (dash-dot). Labels give the share of copy bandwidth used under the minimal traffic model. Temporally blocked variants ("xx") sit above the single-step roof while using less bandwidth, because their roof is k times higher.

the Python counterpart of our single-step kernels. We run it as a user would (`default`: runtime-sized fields, default block size) and tuned (`tuned`: shape fixed at code generation, block size picked by timing five candidates). *AN5D* [11] is the temporal-blocking counterpart. Its generator needs LLVM/Clang 3.4 and GCC 4.8, so we use the CUDA code it generated for the published artifact (commit `fbebf5f`, 5-point `star2d1r` and 7-point `star3d1r`), substitute our coefficients, and build it with `nvcc` 13.4.92 for sm_86 with the artifact's flags. Its configurations were tuned for P100/V100, not for this GPU; we report the best of the shipped ones (1 in 2D and 1 in 3D in this run) and keep AN5D's own timer. Before timing, every counterpart matches our NumPy oracle after 23 steps on 200^2 and 70^3 grids to 2.4×10^{-7} (largest absolute deviation, data in $[0, 1)$), which is fp32 rounding: the codes differ only in summation order. These numbers come from a short run on smaller grids (4096^2, 256^3) on a GPU shared with other jobs and are provisional; the idle-GPU run `bench/counterparts.py -full` at 16384^2 and 512^3 replaces them.

For a single step (fig. 3), tuned pystencils reaches 37.6 GLUPS in 2D (83% of copy) and 29.1 GLUPS in 3D (65%), and 30.8 and 20.6 GLUPS with its defaults. Our best Python-DSL kernels reach 36.7 (`triton/tile`) and 36.6 GLUPS (`triton/zmarch`). The faster is pystencils in 2D, by 1.03×, and `pyheat3d` in 3D, by 1.26×. pystencils generates one thread per site; in 3D that is the structure of our naive kernels, without the z-marching schedule. With temporal block-

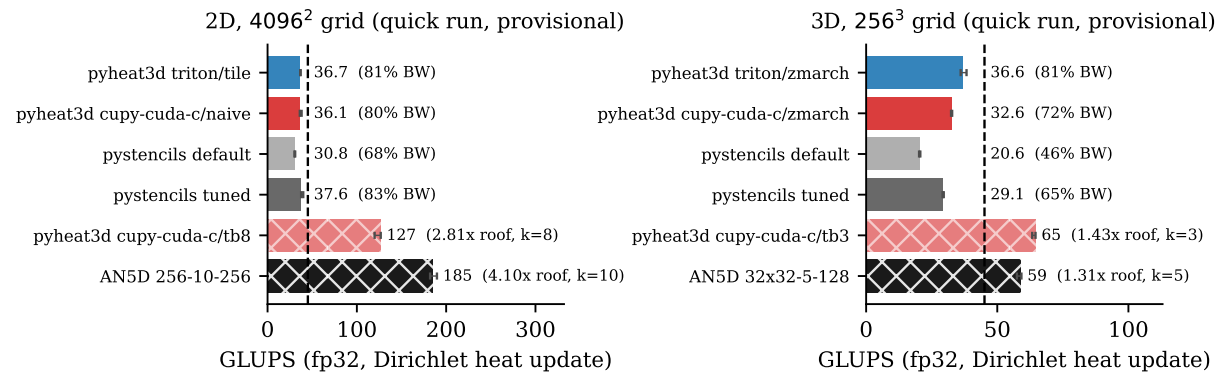


Figure 3: Head-to-head on one GPU in one session: our best Python-DSL and CUDA C single-step kernels, pystencils (grey), our best temporal-blocking kernel and AN5D's best shipped configuration (black). Dashed: single-step fp32 roofline.

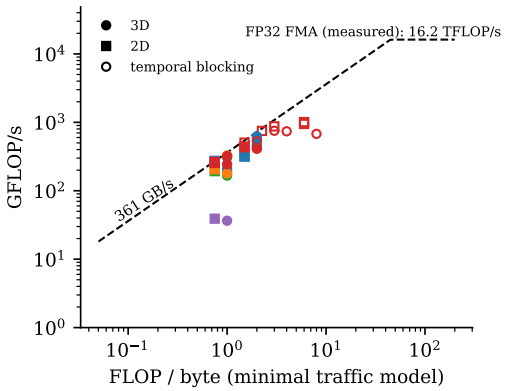


Figure 4: Classic roofline. Intensity uses the minimal traffic model, so temporally blocked kernels (open symbols) sit k times further right.

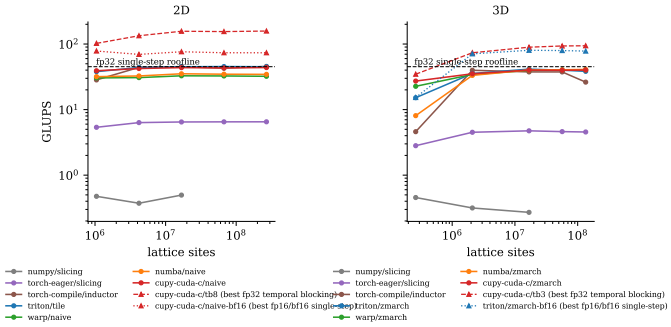


Figure 5: Throughput against problem size for the best fp32 single-step variant of each backend, the best temporally blocked and the best 2-byte variant. Small grids are bound by Python launch overhead and fit in the 4 MB L2.

ing, AN5D's `star2d1r-256-10-256` ($b_T = 10$) reaches 185.1 GLUPS (4.10× the single-step roof) and `star3d1r-32x32-5-128` ($b_T = 5$) 59.0 GLUPS (1.31×), against 126.7 (`cupy-cuda-c/tb8`) and 64.6 GLUPS (`cupy-cuda-c/tb3`) for ours. The faster is AN5D in 2D, by 1.46×, and `pyheat3d` in 3D, by 1.10×. AN5D fuses up to ten steps per pass in 2D where our kernels stop at eight, and was designed to keep register and shared-memory pressure low at such depths [11]. This rung of our ladder is CUDA C launched from Python, so it compares hand-written with generated CUDA, not Python with C.

7 Discussion

Why Python gets there. A memory-bound kernel needs three things: coalesced access, enough bytes in flight, and no redundant passes. Triton's block model gives the first two by construction. `torch.compile` gives the third by fusion. Warp and Numba expose plain SIMT: in 3D, once each thread marches through z with coalesced rows, they match CUDA C; their simple 2D kernels,

Table 2: All variants at the largest benchmarked size. GB/s and % use the minimal traffic model $B_{\mathrm{upd}} = 2b/k$. "equiv.": single-step-equivalent bandwidth LUPS × 8 B (not a DRAM rate).

	backend	variant	store	k	GLUPS	B/upd	GB/s	% copy	% spec	equiv.
2D	`torch-eager`	`slicing`	fp32	1	6.51	8	52	14	12	–
2D	`torch-compile`	`inductor`	fp32	1	45.54	8	364	101	81	–
2D	`triton`	`tile`	fp32	1	45.07	8	361	100	80	–
2D	`triton`	`tile-bf16`	bfp16	1	53.18	4	213	59	47	–
2D	`triton`	`tile-fp16`	fp16	1	52.60	4	210	58	47	–
2D	`warp`	`naive`	fp32	1	32.20	8	258	71	58	–
2D	`numba`	`naive`	fp32	1	34.46	8	276	76	62	–
2D	`cupy-cuda-c`	`naive`	fp32	1	43.76	8	350	97	78	–
2D	`cupy-cuda-c`	`naive-bf16`	bfp16	1	73.56	4	294	82	66	–
2D	`cupy-cuda-c`	`naive-fp16`	fp16	1	71.87	4	287	80	64	–
2D	`cupy-cuda-c`	`ymarch`	fp32	1	42.49	8	340	94	76	–
2D	`cupy-cuda-c`	`tb2`	fp32	2	84.26	4	337	93	75	674
2D	`cupy-cuda-c`	`tb3`	fp32	3	124.82	2.7	333	92	74	999
2D	`cupy-cuda-c`	`tb4`	fp32	4	145.45	2	291	81	65	1164
2D	`cupy-cuda-c`	`tb8`	fp32	8	157.78	1	158	44	35	1262
2D	`cupy-cuda-c`	`tb4-fp16`	fp16	4	167.87	1	168	47	37	671
3D	`torch-eager`	`slicing`	fp32	1	4.55	8	36	10	8	–
3D	`torch-compile`	`inductor`	fp32	1	26.24	8	210	58	47	–
3D	`triton`	`tile`	fp32	1	24.31	8	194	54	43	–
3D	`triton`	`tile-bf16`	bfp16	1	57.19	4	229	63	51	–
3D	`triton`	`tile-fp16`	fp16	1	56.23	4	225	62	50	–
3D	`triton`	`zmarch`	fp32	1	38.33	8	307	85	68	–
3D	`triton`	`zmarch-bf16`	bfp16	1	78.03	4	312	86	70	–
3D	`triton`	`zmarch-fp16`	fp16	1	72.53	4	290	80	65	–
3D	`warp`	`naive`	fp32	1	20.79	8	166	46	37	–
3D	`warp`	`zmarch`	fp32	1	40.19	8	322	89	72	–
3D	`numba`	`naive`	fp32	1	22.42	8	179	50	40	–
3D	`numba`	`zmarch`	fp32	1	40.99	8	328	91	73	–
3D	`cupy-cuda-c`	`naive`	fp32	1	30.57	8	245	68	55	–
3D	`cupy-cuda-c`	`naive-bf16`	bfp16	1	51.45	4	206	57	46	–
3D	`cupy-cuda-c`	`naive-fp16`	fp16	1	51.90	4	208	58	46	–
3D	`cupy-cuda-c`	`zmarch`	fp32	1	40.06	8	321	89	72	–
3D	`cupy-cuda-c`	`tb2`	fp32	2	66.27	4	265	73	59	530
3D	`cupy-cuda-c`	`tb3`	fp32	3	94.10	2.7	251	70	56	753
3D	`cupy-cuda-c`	`tb4`	fp32	4	92.47	2	185	51	41	740
3D	`cupy-cuda-c`	`tb4-fp16`	fp16	4	84.81	1	85	23	19	339

without that schedule, stay at 71–76%. Inductor's generic 3D schedule (58%) shows the same point from the other side: the schedule matters, not the language. The interpreter costs microseconds per launch and nothing per byte. What Python does *not* give for free are the explicit shared-memory and warp-shuffle schedules that temporal blocking needs. Triton deliberately hides them. For that rung we wrote CUDA C, but compiled and launched it from Python.

Threats to validity. (i) One laptop GPU: absolute numbers do not transfer, but percentages of the measured copy bandwidth should transfer better. (ii) Laptop GPUs throttle with temperature and power. Each run records SM clock, temperature and power, and the final benchmark runs serially on an idle GPU. (iv) The CUDA C reference is compiled by NVRTC, not `nvcc`. Both use the same NVVM and ptxas pipeline, but default flags differ. (v) The minimal traffic model ignores halo re-reads, and on small grids part of the working set stays in the 4 MB L2. Effective bandwidths for small grids can therefore exceed what DRAM alone could deliver. We base claims on the largest grids. (vi) Tile shapes were tuned briefly on a shared GPU and are unlikely to be optimal.

8 Conclusion

For the canonical memory-bound stencil, Python reaches the hardware ceiling: in 2D `torch.compile` and Triton, in 3D z-marching kernels in Numba, Warp and Triton run at the level of hand-written CUDA C, close to the copy bandwidth. Where a Python route falls short, it lacks a schedule, not a language feature. Beyond that ceiling only algorithmic change helps. Temporal blocking raises the update rate above the single-step roofline by cutting traffic per update. We report it as a change of roofline, with its traffic, not as a victory over the hardware.

Code availability

Code, benchmark JSON, figure scripts and this paper: https://github.com/walidhemati/pyheat3d (MIT licence). `bench/run.py -full` regenerates every number, and `figures/make_figures.py` regenerates every figure and the macro file this paper is typeset from.

References

[1] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785.

[2] Kaushik Datta, Mark Murphy, Vasily Volkov, Samuel Williams, Jonathan Carter, Leonid Oliker, David Patterson, John Shalf, and Katherine Yelick. Stencil computation optimization and auto-tuning on state-of-the-art multicore architectures. In *Proceedings of the 2008 ACM/IEEE Conference on Supercomputing (SC08)*, pages 1–12, 2008. doi: 10.1109/SC.2008.5222004.

[3] Paulius Micikevicius. 3D finite difference computation on GPUs using CUDA. In *Proceedings of the 2nd Workshop on General Purpose Processing on Graphics Processing Units (GPGPU-2)*, pages 79–84, 2009. doi: 10.1145/1513895.1513905.

[4] Anthony Nguyen, Nadathur Satish, Jatin Chhugani, Changkyu Kim, and Pradeep Dubey. 3.5-D blocking optimization for stencil computations on modern CPUs and GPUs. In *2010 ACM/IEEE International Conference for High Performance Computing, Networking, Storage and Analysis (SC10)*, pages 1–13, 2010. doi: 10.1109/SC.2010.2.

[5] Justin Holewinski, Louis-Noël Pouchet, and P. Sadayappan. High-performance code generation for stencil computations on GPU architectures. In *Proceedings of the 26th ACM International Conference on Supercomputing (ICS '12)*, pages 311–320, 2012. doi: 10.1145/2304576.2304619.

[6] Martin Bauer, Johannes Hötzer, Dominik Ernst, Julian Hammer, Marco Seiz, Henrik Hierl, Jan Hönig, Harald Köstler, Gerhard Wellein, Britta Nestler, and Ulrich Rüde. Code generation for massively parallel phase-field simulations. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC19)*, pages 1–32, 2019. doi: 10.1145/3295500.3356186.

[7] Martin Bauer, Sebastian Eibl, Christian Godenschwager, Nils Kohl, Michael Kuron, Christoph Rettinger, Florian Schornbaum, Christoph Schwarzmeier, Dominik Thönnes, Harald Köstler, and Ulrich Rüde. waLBerla: A block-structured high-performance framework for multiphysics

simulations. *Computers & Mathematics with Applications*, 81:478–501, 2021. doi: 10.1016/j.camwa.2020.01.007.

[8] David Wonnacott. Using time skewing to eliminate idle time due to memory bandwidth and network limitations. In *Proceedings of the 14th International Parallel and Distributed Processing Symposium (IPDPS 2000)*, pages 171–180, 2000. doi: 10.1109/IPDPS.2000.845979.

[9] Gerhard Wellein, Georg Hager, Thomas Zeiser, Markus Wittmann, and Holger Fehske. Efficient temporal blocking for stencil computations by multicore-aware wavefront parallelization. In *2009 33rd Annual IEEE International Computer Software and Applications Conference (COMPSAC)*, pages 579–586, 2009. doi: 10.1109/COMPSAC.2009.82.

[10] Tareq Malas, Georg Hager, Hatem Ltaief, Holger Stengel, Gerhard Wellein, and David Keyes. Multicore-optimized wavefront diamond blocking for optimizing stencil updates. *SIAM Journal on Scientific Computing*, 37(4):C439–C464, 2015. doi: 10.1137/140991133.

[11] Kazuaki Matsumura, Hamid Reza Zohouri, Mohamed Wahib, Toshio Endo, and Satoshi Matsuoka. AN5D: Automated stencil framework for high-degree temporal blocking on GPUs. In *Proceedings of the 18th ACM/IEEE International Symposium on Code Generation and Optimization (CGO 2020)*, pages 199–211, 2020. doi: 10.1145/3368826.3377904.

[12] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, et al. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020. doi: 10.1038/s41586-020-2649-2.

[13] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, et al. PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*, pages 929–947, 2024. doi: 10.1145/3620665.3640366.

[14] Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL 2019)*, pages 10–19, 2019. doi: 10.1145/3315508.3329973.

[15] Miles Macklin. Warp: A high-performance python framework for GPU simulation and graphics. NVIDIA GPU Technology Conference (GTC), https://github.com/nvidia/warp, March 2022.

[16] Siu Kwan Lam, Antoine Pitrou, and Stanley Seibert. Numba: A LLVM-based Python JIT compiler. In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC (LLVM '15)*, pages 1–6, 2015. doi: 10.1145/2833157.2833162.

[17] Ryosuke Okuta, Yuya Unno, Daisuke Nishino, Shohei Hido, and Crissman Loomis. CuPy: A NumPy-compatible library for NVIDIA GPU calculations. In *Proceedings of Workshop on Machine Learning Systems (LearningSys) in the Thirty-first Annual Conference on Neural Information Processing Systems (NIPS)*, 2017. URL http://learningsys.org/nips17/assets/papers/paper_16.pdf.